1. Overview

This document explains how three important system tables work and how they support system operations. Each table has a different purpose, but together they help organize broadcast data, track server activity, and store user or system feedback.

- Hosts track servers, and when they last sent alerts.
- Channels store information about broadcast channels and link them to their markets and affiliates.
- Comments store written feedback from users or system processes.

2. Hosts Table Analysis

Purpose

- The Hosts table records the servers that run background tasks and send alerts. It stores basic details about each server so the system can monitor performance and activity.

Table Structure

Column Name	Data Type	Description
Host_Name	VARCHAR	Server Name(Primary Key).
IP	VARCHAR	IP address of the host.
NoCheck	TINYINT	Marks if the host should skip checks
Last_Sent	DATETIME	The time the host last sent an alert or message.

Columns in table

Column	Type	Nullable	Indexes
Host_Name	varchar(30)	NO	PRIMARY, Host_Name_UNIQUE
IP	varchar(45)	NO	IP_UNIQUE
NoCheck	tinyint	YES	
Last_Sent	datetime	YES	

Keys

- Primary Key: Host_Name
- No foreign keys

Observations

- The table only stores server information and does not connect to user tables.
- It is useful for checking whether servers are working correctly and sending alerts on time.

Recommendations

- Add a host_name field in alert-related tables to track which server sent each alert.

3. Channels Table Analysis

Purpose

- The Channels table keeps information about each broadcast channel, including its name, IP address, market, and affiliate. It helps connect channels to the correct geographic market and network.

Table Structure

Column Name	Data Type	Description
Channel_Id	INT UNSIGNED	The unique ID number for each channel.
Market_Id	INT	Links each channel to a record in the Markets table.
Channel_Name	VARCHAR(20)	The name of the channel (ex: KDVR or KCEC).
Host_Name	VARCHAR(20)	The network or server name for the channel.
IP	VARCHAR(45)	The IP address linked to the channel.
Affiliate	VARCHAR(45)	Connects the channel to its affiliate network (from the Affiliates table)
Virtual	VARCHAR(4)	The virtual number of the channel
Call_Sign	VARCHAR(4)	The call sign that identifies the station publicly

Columns in table

Column	Type	Nullable	Indexes
Channel_Id	int unsigned	NO	PRIMARY, Channel_Id_UNIQUE
Market_Id	int	NO	Market_
Channel_Name	varchar(20)	NO	Market_
Host_Name	varchar(20)	NO	Market_, _idx
IP	varchar(45)	YES	
Affiliate	varchar(45)	YES	fk_Affiliates_idx
Virtual	varchar(4)	YES	
Call_Sign	varchar(4)	YES	

Keys

- Primary Key: Channel_Id
- No foreign keys

Observations

- Some Affiliate values in Channels do not match any records in the Affiliates table.

- All Market_Id values correctly match the Markets table.
- Several channel names appear more than once, which may indicate subchannels or duplicate entries.

Recommendations

- Review and correct Affiliate values that do not match the Affiliates table.
- Check duplicate channel names to confirm whether they are valid or errors.

4. Comments Table Analysis

Purpose

- The Comments table stores written comments from users or the system. It also tracks where the comment came from, when it was created or updated, and whether it is still active.

Table Structure

Column Name	Data Type	Description
email	VARCHAR(100)	The email address of the user who made the comment.
createdAt	TIMESTAMP	The date and time when the comment was first created
commentText	TEXT	The actual content of the comment
appSource	ENUM	The platform where the comment was made
updatedAt	TIMESTAMP	The date and time when the comment was last updated.
isActive	TINYINT	A flag showing if the comment is active (1 = active, 0 = inactive)

Columns in table

Column	Type	Nullable	Indexes
email	varchar(100)	NO	PRIMARY
createdAt	timestamp	NO	PRIMARY
commentText	mediumtext	NO	ft_comment_text
appSource	enum('search engine',...	NO	
updatedAt	timestamp	YES	
isActive	tinyint(1)	YES	

Keys

- Primary Key: email, createdAt
- No foreign keys

Observations

- The table structure is complete and uses appropriate data types.
- The full-text search index speeds up searches inside commentText.

Recommendations

- Make sure all emails in Comments exist in the users table.
- Confirm that timestamps auto-fill correctly for accuracy.
- Archive or delete inactive comments older than two years to improve performance.

5. Overall Recommendations

Data Quality

- Use foreign keys more consistently to prevent broken links, especially Affiliates and user emails.
- Remove or fix orphaned records to maintain accuracy.

Performance

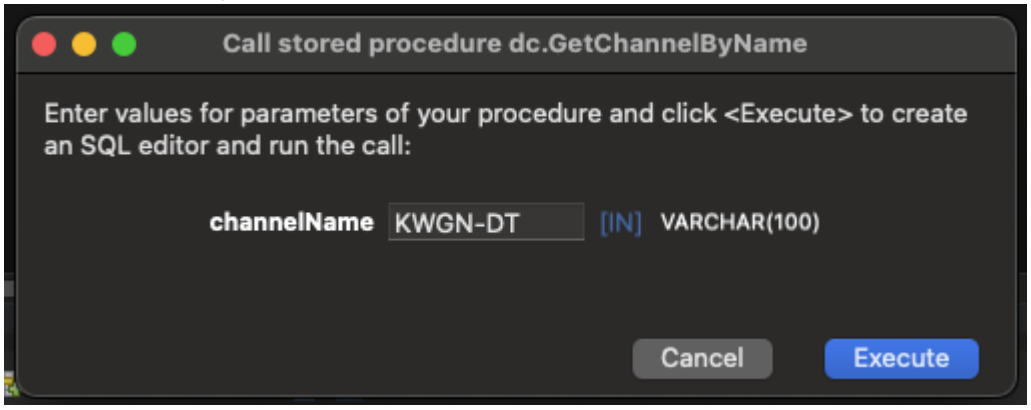
- Add indexes to important columns.
- Archive older records from large tables like Comments to keep the system fast.

System Monitoring

- Connect the Hosts table more directly with alert tables to better track which server handled each alert.

6. Stored Procedures

GetChannelByName

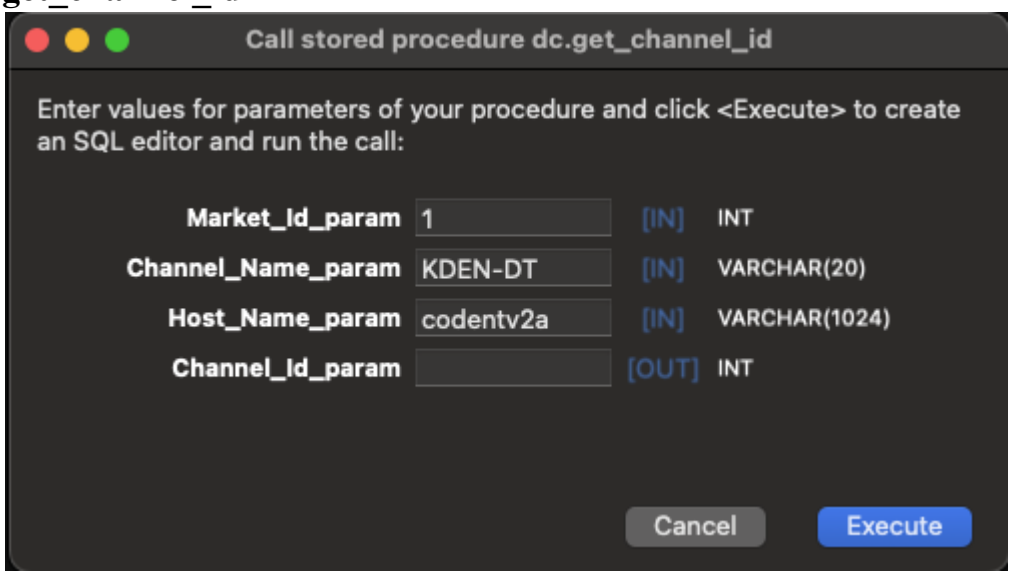


19 17:03:55 call dc.GetChannelByName('KWGN-DT') Error Code: 1146. Table 'dc.channels' doesn't exist

```
CREATE DEFINER=`henry_VPN`@`10.8.0.%` PROCEDURE `GetChannelByName`(IN channelName VARCHAR(100))
BEGIN
    SELECT Channel_Id, Market_Id, Channel_Name, Host_Name, IP, Affiliate, Call_Sign
    FROM channels
    WHERE Channel_Name = channelName;
END
```

- Purpose:
  - Use the channel name to extract that entry in the Channels table
- Error:
  - The sql query is written to pull from the channels table, rather than the Channels table
- Recommendation:
  - Change the ‘channels’ table to ‘Channels’

get_channel_id



```
set @Channel_Id_param = 0;
call dc.get_channel_id(1, 'KDEN-DT', 'codentv2a', @Channel_Id_param);
select @Channel_Id_param;
```

1:1

esult Grid Filter Rows: Search Export:

@Channel_Id_param
157

```
set @Channel_Id_param = 0;
call dc.get_channel_id(1, 'KDEN-DM', 'codentv2a', @Channel_Id_param);
select @Channel_Id_param;
```

35:2

esult Grid Filter Rows: Search Export:

@Channel_Id_param
0

- Purpose:
  - Allows the user to enter the Market_Id, Channel_Name, and Host_Name so the stored procedure can return the correct Channel_Id
- Output:
  - Successful extraction of the Channel_Id
- Recommendation:
  - When the Channel_Id doesn't exist, it just returns it as 0
  - Create a different output that indicates more clearly it is an error

Get_Channels

```
1  call dc.Get_Channels();
2
```

0% 1:2

Result Grid Filter Rows: Search

Channel_Name
KCEC
KCEC-DT
KCNC-TV
KDEN-DM
KDEN-DT
KDVR-DT
KMGH-TV
KTVD-DT
KTVD-HD
KUSA-HD
KWGN-DT

- Purpose:
  - Return a list of all channel names from the Channels table so users can view available broadcast channels.
- Output:
  - The stored procedure successfully returns a list of channel names, confirming that it is pulling data correctly.
- Recommendations:
  - The stored procedure is working as expected and returning the correct channel list.

get_channels_with_data

```
1  call dc.get_channels_with_data();
2
```

00% 1:1

Result Grid Filter Rows: Search

Channel_Name	Virtual

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `get_channels_with_data`(
-- Ask Bob / Henry if they want all channels with active data or specify criteria instead
)
BEGIN
  SELECT DISTINCT Channels.Channel_Name, Channels.`Virtual` FROM Channels JOIN SRT ON Channels.Channel_Id = SRT.Channel_Id;
END
```

SRT - Table | dc.SRT | dc.SRT | Query 15

Info | Columns | Indexes | Triggers | Foreign keys | Partitions | Grants | DDL

Name	Schema	Table	Column	Referenced Sch...	Referenced Table	Referenced Colu...

Limit to 1000 rows

```
SELECT * FROM dc.SRT;
```

100% 1:1

Result Grid Filter Rows: Search Edit: Export/Import:

Channel_Id	Mp4_File_Name	CC_Num	Timecode	Line1	Line2	Line3	Line4	Created_at	Finished_at	Six_frames
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Purpose:
  - Return all channels that have corresponding data in the SRT table by joining Channels to SRT and selecting Channel_Name and Virtual.
- Error:
  - The stored procedure returns 0 rows because the SRT table is empty.
  - Since the procedure uses an INNER JOIN, no rows from Channels will match, resulting in an empty result set.
- Recommendations:
  - Insert sample or real SRT records
  - Add a foreign key of channel_id to the SRT table

Insert_channel

Call stored procedure dc.Insert_channel

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

Channel_Name_param test [IN] VARCHAR(20)

Cancel Execute

```
SELECT * FROM dc.Channels;
```

100% 1:1

Result Grid Filter Rows: Search Edit: Export/Impor

Channel_Id	Market_Id	Channel_Name	Host_Name	IP	Affiliate	Virtual	Call_Sign
207	1	KDEN-DM	codentv2a	NULL	NULL	NULL	NULL
208	1	test	localhost	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Purpose:
  - Insert a new channel into the Channels table using the provided Channel_Name
- Output:
  - The stored procedure successfully inserted a new row into the Channels table:
    - A new Channel_Id was generated (208)
    - Channel_Name was set to "test"
    - Default values were assigned for missing fields (Market_Id = 1, Host_Name = 'localhost')
    - All other columns remain NULL until updated
- Recommendations:
  - Add validation to prevent duplicate channel names

InsertComment

```
call dc.InsertComment('test@gmail.com', 'This is a test', 'website');
```

100% 1:1

ction Output

	Time	Action	Response
1	18:46:11	call dc.InsertComment('test@gmail.com', 'This is a test', 'website')	Error Code: 1449. The user specified as a definer ('henry_VPN'@'%') does not exist

Call stored procedure dc.InsertComment

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

p_email	test@gmail.com	[IN]	VARCHAR(100)
p_commentText	This is a test	[IN]	MEDIUMTEXT
p_appSource	website	[IN]	VARCHAR(20)

Cancel Execute

Name: InsertComment

```sql
CREATE DEFINER=`henry_VPN`@`%` PROCEDURE `InsertComment`(
    IN p_email VARCHAR(100),
    IN p_commentText MEDIUMTEXT,
    IN p_appSource VARCHAR(20)
)
BEGIN
    -- Insert the comment
    INSERT INTO Comments (
        email,
        commentText,
        appSource
    ) VALUES (
        p_email,
        p_commentText,
        p_appSource
    );

    -- Return the created comment
    SELECT email, createdAt
    FROM Comments
    WHERE email = p_email
    ORDER BY createdAt DESC
    LIMIT 1;
END
```

- Purpose:
  - Insert a new comment into the Comments table using the provided email, comment text, and app source.
- Output:
  - Error Code: 1449. The user specified as a DEFINER ('henry_VPN'@'%') does not exist
- Recommendations:
  - Re-evaluate whether the DEFINER is required
  - After fixing the DEFINER, test the procedure again

7. Conclusion

- The Hosts, Channels, and Comments tables all work correctly and support different parts of the system, but each table has areas that can be improved.
- The Hosts table does its job of tracking server activity, but it would work even better if it were more directly connected to alert-related features so the system can see which server sends which alerts.
- The Channels table is well organized, but it contains some issues, such as incorrect Affiliate values and repeated channel names. Cleaning these up will make the data more accurate and easier to maintain.
- The Comments table stores feedback correctly and utilizes a helpful text index, but it requires stronger validation checks to ensure emails are valid and improved cleanup of old or inactive comments.
- Most stored procedures work, but several need fixes such as correcting table name case, adding better error messages, preventing duplicates, and fixing the missing DEFINER user. Updating these procedures will make them easier to run without errors.
- Making these changes will improve data quality, reduce mistakes, and help the system run more smoothly across all tables and procedures.
- Together, these improvements will help ensure that the system remains accurate, efficient, and easier to maintain over time.